Executive Summary — Unexpected Google Places "Nearby Search" Usage

Prepared for: Google Cloud Billing Support (supporting documentation) **Date:** 2026-07-06 **Product/SKU:** Places API — `google.places.NearbySearch.Http` **Application:** Pre-launch real-estate web application (not yet publicly released)

What happened

Between approximately **2026-06-30 and 2026-07-06**, our project recorded roughly **38,236** `NearbySearch` requests, including a spike of approximately **21,702 requests on 2026-07-05** concentrated in a few short bursts over several hours.

The application had **not been publicly launched**. These requests were **not** produced by end users. Our internal investigation determined they were generated by our **automated test suite** during a day of intensive feature development, which unintentionally executed live API calls instead of mocked ones.

Root cause (summary)

A feature that enriches property listings with nearby points of interest calls the Places `NearbySearch` endpoint **16 times per listing** (once per category). During automated testing, four conditions combined so that these calls hit the **live** API instead of a test stub:

1. The test environment ran jobs **synchronously** (a dispatched background task executed immediately, inline).
2. A valid API key was **present in the test environment** and was not overridden to a blank/test value.
3. The code path used a low-level HTTP client that our standard test-mocking tool **could not intercept**.
4. The test suite had **no global network guard**, so nothing prevented outbound calls.

Because each automated test run resets its database and cache, **no caching applied**, so every test iteration re-issued the full set of requests. Running the affected tests repeatedly during development produced the burst pattern seen on the bill.

We independently **measured** the per-listing cost (16 requests/generation) offline, with **zero live calls**, and confirmed the mechanism. Full technical detail, evidence, and confidence levels are in the accompanying Root Cause Analysis.

Why this was not caught sooner

The relevant code did not log or meter its outbound requests, and no budget alert, quota cap, or key restriction was configured. The unusual volume was therefore first noticed on the **billing report**, not in our monitoring.

What is not in dispute vs. still being quantified

- **Confirmed:** the traffic originated from our own automated tests during development, not from production users, a data import, a scheduled job, or malicious third-party use.
- **Being quantified:** because our test environment used the live key, a portion of the 21,702 requests may have returned `REQUEST_DENIED` (invalid-key responses that still register as requests but are typically non-billable) versus successful billable responses. We cannot split this from our side; **Google's per-response metrics are the authoritative source** for the billed subset.

Containment and prevention (in progress)

- [DONE] **The Places API has been disabled** in the affected project to stop any further usage.
- [PLANNED] Test environment will be hardened to force a **blank/stub key** and block all outbound calls during testing.
- [PLANNED] The application will add a **daily request cap and outbound-request logging**.
- [PLANNED] In Google Cloud we will add **API-key restrictions, a NearbySearch quota cap, and a budget alert** before the API is re-enabled.

Request

We respectfully request review of the charges associated with this **unintentional, development-generated, non-production** usage, given that it resulted from a testing misconfiguration rather than genuine application traffic, has been contained, and is being permanently remediated.

This summary accompanies a detailed Root Cause Analysis available on request. Prepared internally; no production code was modified during the investigation.